# Trend Leader — Windows 신규 개발자 환경 구축 가이드

> 문서 버전: v1.0 작성일: 2026-10-14 대상: Windows 10/11 + PowerShell 신규 개발 참여자 기준 저장소: `Trend-Leader-capstoneProject/feature` 기준 브랜치: `dev` (작성 시점의 설정 파일 기준이며, 실제 착수 시 최신 HEAD 확인 필요) 목적: 받은 소스를 **개인 PC에서 실행·검증**한 뒤 기능별 작업 브랜치에서 개발할 수 있도록 준비 상태: **팀 배포용 가이드 초안** — 신규 팀원 PC에서 실제 수행한 결과는 아직 확인하지 않음

---

## 빠른 시작 (이미 Git Pull과 `.env` 전달을 마친 팀원용)

아래는 **전체 절차를 한 번에 보는 요약**입니다. 설치가 안 되어 있거나 명령이 실패하면 **해당 단계에서 멈추고 아래 상세 설명**을 참고하세요. `docker-compose.yml`이 있는 저장소 루트의 PowerShell에서 시작합니다.

```
# 1) 현재 작업 위치 / 브랜치 / 받은 환경파일 확인
Test-Path .\docker-compose.yml
git status -sb
Test-Path .\.env.compose
Test-Path .\backend\.env
Test-Path .\frontend\.env

# 2) Docker 엔진 및 실행
# (환경파일이 올바르게 준비되고 Docker Desktop이 실행 중이어야 함)
docker version
docker compose --env-file .env.compose config
docker compose --env-file .env.compose up --build -d
docker compose --env-file .env.compose ps

# 3) 본인의 신규 로컬 DB에 스키마/초기 카테고리 반영
docker compose --env-file .env.compose exec backend python -m alembic upgrade head
docker compose --env-file .env.compose exec backend python -m
scripts.seed_categories

# 4) API + DB 정상 동작 확인
Invoke-RestMethod http://localhost:8000/api/health/ready
```

새 PowerShell에서 Frontend를 실행합니다.

```
node --version  # package.json 요구: >=22.13.0 <23
cd frontend
npm ci
npx expo start -c
```

이후 Swagger(<http://localhost:8000/docs>)와 Android 앱에서 API 연결을 확인합니다. 외부 Backend 포트를 변경했다면 위 접속 주소 및 `frontend/.env`도 그 포트에 맞춰야 합니다. **실행만 했다고 온보딩 완료로 판정하지 않습니다.**

> 보안: 전달받은 환경파일을 덮어쓰지 않기 / 실제 비밀번호·Secret·`docker compose config` 전체 결과 공유하지 않기 / `down -v` 실행하지 않기.

---

# 0. 먼저 읽기: 이번 온보딩의 완료 기준

이 가이드는 새로운 기능을 구현하는 문서가 아닙니다. 아래 항목이 **본인 PC에서** 확인되면 환경 구축을 완료한 것으로 판단합니다.

- [ ] GitHub `feature` 저장소의 최신 `dev` 코드를 받았고, 로컬 변경사항을 확인했다.
- [ ] Windows에서 Docker Desktop의 Linux Engine과 Docker Compose가 동작한다.
- [ ] 전달받은 환경변수 파일이 **올바른 위치와 이름**으로 존재한다.
- [ ] MariaDB(`db`)와 FastAPI(`backend`) 컨테이너가 실행된다.
- [ ] Alembic Migration과 Category Master Seed가 정상 실행된다.
- [ ] `/api/health/ready`에서 `data.database = "available"`를 확인했다.
- [ ] Swagger(`/docs`)에 접속할 수 있다.
- [ ] Frontend 의존성을 설치하고 Expo 앱을 실행할 수 있다.
- [ ] 실기기 또는 에뮬레이터에서 기존 API 연동 흐름을 확인했다.
- [ ] 공통 개발 가이드와 Git 브랜치/PR 방식의 위치를 알고 있다.

**기본 개발 실행 방식(권장)**

```
Windows PC
├─ Docker Desktop / Docker Compose
│  ├─ MariaDB 컨테이너: db (내부 3306 / 기본 호스트 3306)
│  └─ FastAPI 컨테이너: backend (내부 8000 / 기본 호스트 8000)
│
└─ Windows PowerShell
   └─ frontend/ → npm → Expo 개발 서버
      └─ Android 실기기 또는 에뮬레이터
```

첫날에는 **Backend와 DB는 Docker에서**, **Frontend는 Windows에서** 실행합니다. Windows에서 별도 MariaDB 서버를 중복 설치할 필요는 없습니다. Python 가상환경과 Backend 테스트 준비는 9절에 따로 설명합니다.

---

# 1. 지금까지 진행한 작업 확인

현재 공유된 상황: Codex, Notion, GitHub 연결 완료 / 저장소 `git pull` 완료 / 팀에서 로컬 환경변수 파일 전달 완료.

**이미 받은 `.env`를 무조건 다시 만들거나 덮어쓰지 마세요.** 환경변수는 Git에서 제외되는 개인 설정이고, 파일명·위치가 실행 방식마다 다릅니다.

## 1.1 저장소 및 브랜치 확인

**PowerShell에서 `docker-compose.yml`이 보이는 저장소 루트로 이동**한 뒤 실행합니다.

```
Get-Location
Test-Path .\docker-compose.yml
```

```
git remote -v
git status -sb
git branch --show-current
git log -1 --oneline
```

`Test-Path`가 `True`여야 하며, `git remote -v`의 `origin`이 팀의 `feature` 저장소인지 확인합니다. `main`이 아닌 ** 개발 통합 브랜치 `dev`**를 기준으로 작업을 시작합니다.

로컬 변경사항이 없다면 다음을 실행할 수 있습니다.

```
git fetch origin
git switch dev
git pull --ff-only origin dev
```

- `git switch dev`가 실패하면 브랜치 존재 여부를 먼저 확인합니다. 다른 브랜치에서 작업 중인 변경사항이 있으면 임의로 폐기하지 않습니다.
- `git pull`을 이미 했더라도 **어느 브랜치에서 받았는지**는 별개이므로 확인해야 합니다.
- 개발은 `dev`에 직접 커밋하지 않고, 담당 기능이 정해지면 최신 `dev`에서 기능 브랜치를 생성합니다.

---

## 2. Windows 도구 사전 점검

| 도구 | 이번 설정에서 필요한 이유 | 확인 명령 |
|---|---|---|
| Git | 브랜치, Pull, PR 협업 | `git --version` |
| Docker Desktop | MariaDB/FastAPI 실행 | `docker version` |
| Docker Compose | 컨테이너 묶음 실행 | `docker compose version` |
| Node.js 22.x | `frontend/package.json`의 `>=22.13.0 <23` 요구조건 | `node --version` |
| npm | Frontend 의존성 설치 | `npm --version` |
| VS Code 또는 기존 편집기 | 소스 탐색 및 터미널 | 실행 여부 확인 |
| Android 실행 수단 | Expo 앱 검증 | Android 실기기 또는 Android Studio 에뮬레이터 |
| Python 3.12 | **선택**: Windows에서 Backend 코드 직접 실행·테스트 시 | `python --version` |

**버전 참고**: 현재 `backend/Dockerfile`은 `python:3.12-slim`을 사용합니다. 저장소 README 일부 표기에는 Python 3.14 배지가 있지만, 이 가이드의 Docker 경로는 **실제 Dockerfile의 Python 3.12**에 맞춥니다. Frontend는 `package.json`의 Node.js 엔진 조건을 우선합니다.

### 2.1 Docker Desktop 시작

1. Docker Desktop을 설치하고 실행합니다.
2. Linux Container Engine이 정상 시작될 때까지 기다립니다.
3. 설치·실행이 막히면 WSL 상태를 확인합니다.

```
wsl --status
docker version
docker compose version
```

- `docker version`은 **Client뿐 아니라 Server/Engine 정보도** 확인되어야 합니다.
- WSL2/가상화 관련 오류가 있다면 Docker Desktop의 Windows 설치 안내에 따라 해당 PC의 Windows 기능·BIOS 가상화·WSL2 구성을 점검합니다. 다른 프로그램이나 OS 설정을 무조건 초기화하지 않습니다.
- 처음 설치하거나 Windows 기능을 변경했으면 재부팅이 필요할 수 있습니다.

공식 안내: Docker Desktop Windows 설치

---

# 3. 전달받은 환경변수 파일 점검 — 덮어쓰기 금지

## 3.1 정확한 파일 위치

프로젝트 루트에서 아래 명령을 실행합니다.

```
Test-Path .\.env.compose
Test-Path .\backend\.env
Test-Path .\frontend\.env
```

| 대상 파일 | 사용처 | 필수 여부 |
|---|---|---|
| `.env.compose` | Docker Compose의 DB 이미지, DB 계정, 노출 포트 | Docker 방식에서 필수 |
| `backend/.env` | FastAPI, JWT 등 서버 설정 | Docker 방식에서 필수 |
| `frontend/.env` | Expo 앱의 API 포트/접속 경로 | Frontend 실행 전 확인 권장 |

> **중요**: 루트 `.env`, `.env.compose`, `backend/.env`는 서로 다른 파일입니다. 받은 파일이 단순히 `env`라는 이름이거나 루트 `.env` 한 개뿐이라면, 팀원에게 원래 의도된 파일명을 확인해야 합니다.

**파일이 없을 때에만** 저장소의 예시 파일에서 생성할 수 있습니다.

```
if (!(Test-Path .\.env.compose)) { Copy-Item .\.env.compose.example .\.env.compose
}
if (!(Test-Path .\backend\.env)) { Copy-Item .\backend\.env.example .\backend\.env
}
if (!(Test-Path .\frontend\.env)) { Copy-Item .\frontend\.env.example
.\frontend\.env }
```

생성만으로 설정이 완료되는 것은 아닙니다. 실제 계정 정보와 필수값을 팀의 로컬 개발환경에 맞게 채웁니다.

### 3.2 값이 맞아야 하는 항목

`.env.compose`

```
# 형식 예시 — 실제 전달받은 비밀번호로 대체(문서/채팅에 실값 공유 금지)
MARIADB_IMAGE_TAG=latest
DB_NAME=trend_leader
DB_USER=trend_user
DB_PASSWORD=<local-db-password>
DB_ROOT_PASSWORD=<local-root-password>
DB_EXTERNAL_PORT=3306
BACKEND_EXTERNAL_PORT=8000
```

`MARIADB_IMAGE_TAG=latest`는 현재 예시 파일 값입니다. 태그를 고정하지 않아 팀원 PC마다 설치 시점에 다른 MariaDB 이미지가 내려올 위험이 있으므로, **팀에서 테스트한 태그로 맞추는 작업은 별도 협의 사항**입니다. 이미 전달받은 값을 멋대로 변경하지 않습니다.

**`backend/.env` 확인 항목**

- `JWT_SECRET_KEY`: 현재 `backend/app/core/config.py` 기준 **32자 이상 필수**. 실제 값은 공유 문서에 복사하지 않습니다.
- `DATABASE_URL=`: Docker Compose 실행 시 **비운 상태로 유지**하는 설정입니다. 값이 있다면 `DB_*` 설정보다 우선할 수 있습니다.
- Docker Compose는 Backend에 `DB_HOST=db`, `DB_PORT=3306`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`를 주입하므로, 컨테이너와 호스트에서 Backend를 직접 실행할 때 DB 연결 주소가 다릅니다.
- 프론트엔드 `EXPO_PUBLIC_*`에는 서버 비밀값(JWT Secret, API Secret, DB 비밀번호)을 절대 작성하지 않습니다.

**`frontend/.env` 예시 파일의 현행 기본값**

```
EXPO_PUBLIC_API_PORT=8000
EXPO_PUBLIC_API_PREFIX=/api
```

특별한 이유가 없다면 우선 그대로 사용하고, Backend의 실제 외부 포트를 바꾼 경우 함께 수정합니다.

**보안 확인**

```
git status --short
git check-ignore -v .env.compose backend/.env frontend/.env
```

- 환경변수 파일이 추적되지 않는지 확인합니다. GitHub, 공개 Issue, Codex 채팅, 스크린샷에 실제 Secret 값이나 `docker compose config` 전체 출력 결과를 올리지 않습니다.
- `docker compose ... config` 출력에는 치환된 비밀번호가 포함될 수 있습니다. **각자 PC에서만 확인**하고, 문제 공유 시 비밀번호를 마스킹합니다.

---

# 4. Docker Compose로 Backend + MariaDB 실행

**아래 Docker 명령은 모두 저장소 루트에서 실행합니다.**

## 4.1 Compose 설정 검사

```
docker compose --env-file .env.compose config
```

`variable is not set`, `DB_NAME is required` 등 오류가 없고 서비스 구성이 출력되는지 확인합니다. 비밀번호가 출력될 수 있으므로 결과 전체를 팀 채팅에 붙여넣지 않습니다.

## 4.2 최초 실행

```
docker compose --env-file .env.compose up --build -d
```

- `up`: 서비스를 만들고 실행
- `--build`: 현재 Backend Dockerfile/소스를 포함해 이미지를 빌드
- `-d`: 백그라운드 실행

첫 실행 시 이미지 다운로드/설치에 시간이 걸릴 수 있습니다.

## 4.3 상태 확인

```
docker compose --env-file .env.compose ps
```

기대 상태(로컬 실행으로 직접 확인 필요):

```
trend-leader-db    Up (healthy)
trend-leader-api   Up
```

MariaDB 컨테이너가 `healthy`가 되어야 Backend를 시작하도록 `depends_on`이 설정되어 있습니다. 단, 컨테이너가 `Up`이라는 사실만으로 DB 스키마까지 정상 준비되었다는 뜻은 아닙니다.

## 4.4 로그 확인

```
docker compose --env-file .env.compose logs --tail=100 db backend
```

실시간 로그가 필요하면:

```
docker compose --env-file .env.compose logs -f db backend
```

`Ctrl+C`는 로그 보기 종료이며, 백그라운드 컨테이너 자체는 계속 실행됩니다.

---

# 5. MariaDB Migration 및 Category Seed

**최초 생성한 로컬 개발 DB에서만** 아래를 실행합니다. Alembic/Seed는 DB 상태를 변경하므로 DB가 본인의 로컬 컨테이너를 향하는지 먼저 확인합니다.

현재 `backend/sql_script/init.sql`은 DB 생성용 SQL이며, 전체 테이블 스키마를 대신하지 않습니다. 테이블은 **Alembic Migration**으로 적용합니다.

## 5.1 스키마 최신화

```
docker compose --env-file .env.compose exec backend python -m alembic upgrade head
```

확인용:

```
docker compose --env-file .env.compose exec backend python -m alembic current
```

`current`가 현재 HEAD Migration과 일치하는지 확인합니다. 실패 시 로그를 보고 멈춥니다. `stamp head`로 오류를 숨기거나 DB를 초기화하지 않습니다.

## 5.2 카테고리 초기 데이터

```
docker compose --env-file .env.compose exec backend python -m
scripts.seed_categories
```

정상 완료 시 프로그램 출력 예시:

```
Category Master Seed 생성 및 검증이 완료되었습니다.
```

현재 README 기준으로 빈 DB에서는 **대분류 6개 + 세부분류 38개 = 44개**를 대상으로 합니다. Seed는 API 실행 또는 Migration 과정에서 자동으로 수행되지 않습니다.

**Demo Trend Seed(선택)**: 트렌드 목록 UI 검증이 필요할 때만 작업 담당자와 데이터 범위를 확인한 후 `python -m scripts.seed_demo_trends` 실행을 검토합니다. 온보딩 필수 단계가 아닙니다.

---

# 6. FastAPI / DB 연결 검증

기본 외부 포트가 `8000`일 때 아래 주소를 확인합니다.

| 주소 | 확인 목적 |
|---|---|
| http://localhost:8000/ | FastAPI 프로세스 접근 |
| http://localhost:8000/api/health | 프로세스 건강 상태 |
| http://localhost:8000/api/health/ready | Backend + DB 준비 상태 |
| http://localhost:8000/docs | 실제 등록된 Swagger Endpoint |

PowerShell에서도 확인할 수 있습니다.

```
Invoke-RestMethod http://localhost:8000/api/health/ready
```

정상 기대 결과의 핵심 값:

```
{
  "success": true,
  "data": {
    "status": "ready",
    "database": "available"
  }
}
```

이는 **핵심 필드만 발췌한 예시**입니다. 전체 응답에는 다른 공통 필드가 존재할 수 있습니다. `data.database`가 `available`이면 연결 확인 단계 완료입니다.

Swagger에 접근할 수 있더라도 **전체 기능이 구현되었다는 뜻은 아닙니다.** 실제 표시되는 Endpoint와 동작을 기준으로 판단합니다.

---

# 7. Frontend 설치 및 Expo 실행

## 7.1 Node.js 버전 재확인

```
node --version
npm --version
```

`frontend/package.json`의 명시 조건은 **Node.js `>=22.13.0 <23`**입니다. 버전이 다르면 팀에서 관리하는 Node 버전 관리 방식에 따라 22.x로 맞춥니다.

## 7.2 의존성 설치 및 실행

새 PowerShell 터미널에서 저장소 루트로 이동한 후:

```
cd frontend
npm ci
npx expo start -c
```

- `npm ci`: 저장소의 `package-lock.json`에 맞춰 의존성을 설치합니다. 잠금 파일을 임의로 고치지 않습니다.
- `-c`: Expo/Metro 캐시를 지우고 시작합니다.
- 최초 실행 또는 Expo 실행 방식에 따라 Android Studio 에뮬레이터 / 실기기 설정이 필요할 수 있습니다.
- 현재 `package.json`의 Expo SDK는 **57 계열**입니다. 기기의 Expo Go와 호환되지 않으면 억지로 패키지를 변경하지 말고, 해당 SDK 지원 여부와 **개발 빌드(`expo-dev-client`) 방식**을 팀과 확인합니다.

Frontend 검증 명령(같은 `frontend/` 터미널에서):

```
npm run typecheck
npm test -- --runInBand
```

테스트 성공 여부는 실제 실행 결과로 판단합니다. 초기 의존성 오류가 발생하면 `npm ci`가 실패한 원인을 먼저 확인하고 `package-lock.json`을 삭제하는 방식으로 즉시 우회하지 않습니다.

### 7.3 Android 실기기의 API 접근 주의

- PC의 `localhost`와 **실기기의 `localhost`는 서로 다릅니다.**
- 현재 Frontend 기본 설정은 Expo 개발 서버 Host 기반으로 API 주소를 구성하며, 필요 시 별도의 API Base URL 설정을 사용합니다.
- PC와 실기기가 같은 네트워크에 있는지, 실제 Backend 주소/포트로 접근 가능한지, Windows 방화벽이 연결을 차단하는지 확인합니다.
- Backend 외부 포트가 `8001` 등으로 변경되었다면 `frontend/.env`의 `EXPO_PUBLIC_API_PORT`도 맞춰야 합니다. 환경변수 변경 후 Expo를 재시작합니다.
- 모바일 앱에서 로그인/API 요청까지 실제로 확인해야 통합 연결이 완료된 것입니다.

---

## 8. 프로젝트 이해와 첫 동작 검증

### 8.1 소스 읽기 순서

1. `README.md`: 서비스 목적, 기술 스택, 디렉터리 역할, 실행 명령
2. `docs/prompt/Trend_Leader_AI_Development_Guidelines.md`: 팀의 AI 협업·설계·검증 규칙
3. 해당 기능의 최신 `docs/designs-feature/` 설계 확정안(실제 경로·파일명 확인)
4. `backend/app/api/`에서 해당 기능 Router → Schema → Service → Repository → Model → Test
5. `frontend/src/features/`에서 Screen → Hook → API Function → Client → Provider/Navigation → Test
6. Notion의 API 명세와 Git 협업 Workflow: 코드/설계 차이는 **조용히 임의 수정하지 않고** 팀에 공유

README의 '계획된 기능 목록'이나 과거 완료 현황은 오늘의 구현 증거가 아닙니다. 최신 브랜치에서 실제 Router/Frontend/테스트를 대조합니다.

### 8.2 수동 검증 예시

- [ ] Swagger에서 등록된 API를 확인했다.
- [ ] `GET /api/categories`에서 초기 카테고리 데이터를 확인했다.
- [ ] Android 앱이 실행된다.
- [ ] 기존 회원가입 → 관심사 선택 또는 기존 로그인 흐름의 **현재 구현된 부분**을 실행했다.
- [ ] 오류 발생 시 재현 단계, 명령, HTTP Status, 민감 정보 제거한 로그를 기록했다.

백엔드, DB, 모바일 앱을 연결한 뒤 기능을 실행해야 환경 검증이 완료됩니다.

---

# 9. 선택: 로컬 Python 개발 및 Backend 테스트

**첫날 Docker 기동만 할 목적이라면 Windows에 Python을 반드시 설치할 필요는 없습니다.** 다만 Backend 코드를 수정하거나 pytest/ruff를 반복 실행한다면 별도의 Windows 가상환경이 편리합니다.

현재 Backend Docker 이미지는 `backend/requirements.txt`만 설치하며, `pytest`/`ruff`는 별도 `backend/requirements-dev.txt`에서 관리됩니다. 따라서 **기본 Docker 컨테이너에 `pytest`가 있다고 가정하면 안 됩니다.**

## 9.1 Python 3.12 가상환경(호스트)

프로젝트 루트에서 다음 명령을 실행합니다. Python 3.12가 PC에 등록되어 있다면:

```
cd backend
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pytest --version
```

`.venv`는 로컬 가상환경이며 Git에서 제외됩니다. 가상환경 활성화가 PowerShell 실행 정책에 막혀도 위처럼 `.venv\Scripts\python.exe`를 직접 사용하면 됩니다.

**Backend 서버를 호스트에서 직접 실행하는 경우에는** Docker 컨테이너와 다르게 `backend/.env`의 `DB_HOST`, `DB_PORT`가 **Windows 호스트에서 접근 가능한 DB 주소와 외부 포트**를 가리켜야 합니다. Docker Compose가 주입하는 `DB_HOST=db`는 컨테이너 네트워크의 이름이며 호스트 Python에서는 기본적으로 사용할 수 없습니다.

## 9.2 테스트 DB 분리(권장)

Repository에는 `docker-compose.test.yml`, `.env.test.compose.example`, `backend/.env.test.example`이 별도로 있습니다. 통합 테스트를 수행할 때는 일반 개발 DB와 분리된 **테스트 전용 DB**를 사용해야 합니다.

- 테스트 DB의 접속 계정과 `TEST_DATABASE_URL` 등은 실제 테스트 설정 및 팀 지침을 읽고 맞춥니다.
- 일반 개발 DB를 테스트용으로 지정하지 않습니다.
- 테스트용 Compose 서비스의 기본 호스트 포트는 `3307`이므로 해당 포트의 충돌 여부를 확인해야 합니다.
- 첫날은 Backend의 테스트 DB 전체 구성을 억지로 완료하기보다 **실행 환경 확인을 우선**하고, 담당 기능 개발 전 테스트 설정을 팀과 함께 검증합니다.

---

# 10. 자주 발생하는 오류와 진단 순서

| 증상 | 우선 확인 | 조치 방향 |
|---|---|---|
| `docker` 명령 실행 실패 | Docker Desktop 설치 및 PATH | Desktop 설치/재시작, 새 터미널 열기 |
| Docker Engine pipe 오류 | Desktop/WSL2/Linux Engine 상태 | Engine 정상 시작 후 다시 시도 |
| `variable is not set` | `.env.compose` 존재 및 `--env-file` | 파일명/위치/변수 확인 |
| DB/Backend 컨테이너 기동 실패 | `docker compose ... ps`, `logs` | 최초 오류 메시지부터 확인 |
| `Ports are not available` | 호스트의 3306/8000 사용 여부 | `.env.compose`의 **외부 포트** 조정 후 연결 설정 동기화 |
| JWT 설정 검증 실패 | `backend/.env`의 Secret 길이 | 32자 이상 로컬 Secret 사용 |
| DB 인증 또는 연결 실패 | `DATABASE_URL`, DB 계정, `db` healthy | Compose DB 변수와 Volume 초기화 상태 확인 |
| Migration 실패 | 현재 DB가 로컬 DB인지, Alembic 로그 | 실패 원인 조사, `stamp` 또는 초기화로 은폐 금지 |
| Categories가 비어 있음 | Migration 및 Category Seed 실행 여부 | Seed 명시 실행 |
| Swagger는 보이지만 앱 API 요청 실패 | 실기기 네트워크/PC IP/외부 포트/방화벽 | 실기기에서 서버 도달 가능 여부 확인 |
| `npm ci` 실패 | Node 버전, `package-lock.json`, 원본 오류 | 잠금 파일 보존 후 원인 해결 |
| Expo Go에서 실행 불가 | Expo SDK와 기기 실행 앱 호환 | SDK 지원 또는 dev client 사용 여부 확인 |
| Python `ModuleNotFoundError: app` | 현재 작업 디렉터리와 실행 방법 | `backend/`에서 `python -m scripts.seed_categories` 사용 |

로그 확인 명령(루트):

```
docker compose --env-file .env.compose ps
docker compose --env-file .env.compose logs --tail=100 db backend
```

**파괴적 명령 주의**

- `docker compose --env-file .env.compose down`은 컨테이너를 제거하지만 기본적으로 DB Volume을 보존합니다.
- `docker compose --env-file .env.compose down -v`는 **개발 데이터가 저장된 Volume까지 삭제**합니다. 공유 DB인지, 저장할 데이터가 있는지 확인하지 않았다면 실행하지 마세요.
- 재시작(`restart`)만으로 변경된 Backend Python 코드가 새 이미지에 반영되는 것은 아닙니다. 현재 Compose 설정에 **소스 볼륨 바인드가 없기 때문에**, Backend 파일 변경 후에는 필요에 따라 `up --build -d`로 새 이미지

를 만들어야 합니다.

---

# 11. 일상 실행 명령 요약

모두 **저장소 루트**에서 실행합니다(`frontend` 명령만 해당 폴더에서).

```
# Docker 컨테이너 시작(기존 빌드)
docker compose --env-file .env.compose up -d

# Backend 소스/의존성 변경 시 재빌드
docker compose --env-file .env.compose up --build -d

# 컨테이너 상태
docker compose --env-file .env.compose ps

# 최근 로그
docker compose --env-file .env.compose logs --tail=100 db backend

# DB 스키마 변경사항 반영(대상 확인 후)
docker compose --env-file .env.compose exec backend python -m alembic upgrade head

# 정상적으로 컨테이너 중지 (DB Volume 보존)
docker compose --env-file .env.compose down
```

Frontend는 새 PowerShell에서:

```
cd frontend
npx expo start --go -c
```

---

# 12. Git 협업 및 Codex 안전한 시작

## 12.1 기본 원칙

- 새 기능은 **최신 `dev` 기반의 기능 브랜치**에서 진행합니다. 브랜치 이름은 Notion의 Git Workflow를 확인합니다.
- 구현 전에 문제/범위/설계 확정안/Acceptance Criteria를 확인합니다.
- 개발은 `설계 확인 → 구현 → 자동 테스트 → 실기기/API 검증 → PR`로 진행합니다.
- 개발환경 온보딩 중 Codex에는 **우선 읽기·분석만** 요청합니다. 설치 명령 실행/설정 변경/커밋/푸시는 명시적 승인 이후 수행합니다.
- `.env*` 실제 값이나 로그에 포함된 Access Token·DB 비밀번호를 공유하지 않습니다.

## 12.2 Codex 온보딩용 프롬프트

```
Trend Leader 신규 개발 참여자입니다.
기준은 feature 저장소의 최신 dev 브랜치입니다.
```

```
1) docs/prompt/Trend_Leader_AI_Development_Guidelines.md를 먼저 읽어주세요.
2) 현재 dev의 Docker Compose, Backend Dockerfile, 환경변수 example,
   Alembic, Category Seed, frontend/package.json을 확인해주세요.
3) 이 가이드와 실제 코드가 다르면 차이와 영향을 표로 정리해주세요.
4) Backend Router 및 Frontend Feature를 기준으로 실제 구현 범위를 확인해주세요.
5) 제 PC에서 실행한 명령 결과를 보내면, Secret을 마스킹한 로그만으로
   다음에 확인할 사항을 설명해주세요.

이번 단계에서는 코드를 수정하거나, 패키지를 설치하거나,
파일을 생성·삭제하거나, Git commit/push를 실행하지 마세요.
모르는 사항은 추측하지 말고 '추가 확인 필요'로 표시해주세요.
```

### 12.3 처음 참여한 팀원이 공유할 결과

다음 정보를 팀에 전달하면 첫 작업 배정과 환경 이슈 해결이 쉽습니다.

```
[온보딩 점검 결과]
OS / Windows 버전:
현재 브랜치 및 git log -1:
Docker Desktop/Engine 동작 여부:
db healthy 여부:
backend Up 여부:
Migration current:
Category Seed 결과:
/api/health/ready 결과 (Secret 제거):
Swagger 접근 여부:
Node.js 버전:
Expo 앱 실행 여부:
실기기 API 호출 여부:
미완료 항목과 오류 재현 순서:
```

---

## 13. 참고 자료 / 이 문서의 검증 범위

**저장소의 `dev` 브랜치에서 실제로 확인한 주요 파일(2026-10-14)**

- README.md: 프로젝트 소개, Docker·Frontend 실행, 브랜치 전략
- docker-compose.yml: `db`, `backend`, 포트, 볼륨, Healthcheck
- backend/Dockerfile: Python 3.12 이미지 및 기본 의존성
- backend/app/core/config.py: DB URL 우선순위, JWT Secret 최소 길이
- .env.compose.example, backend/.env.example, frontend/.env.example
- backend/alembic/env.py, backend/scripts/seed_categories.py
- frontend/package.json, backend/requirements-dev.txt
- Trend Leader AI Development Guidelines

**확인 수준 구분**

- **확인됨**: 저장소 소스에 적힌 명령, 파일명, 설정, 실행 흐름.

- **아직 확인하지 않음**: 새 팀원 PC의 Docker/WSL2, 포트 충돌 여부, Node/Python 설치 상태, 실제 DB Migration 성공, Expo/Android 실행 성공, 앱 API 통합 테스트 성공.
- 실행 중 위 가이드와 실제 코드가 불일치하면 **최신 브랜치 코드와 기능별 확정안**을 함께 대조하고 차이를 보고한 후 갱신합니다.

**유지보수 안내**: 이 파일은 신규 참여자의 빠른 환경 재현을 위해 작성한 온보딩 문서입니다. 공통 실행 방법의 Source of Truth는 실제 Compose/Dockerfile/환경변수 example과 프로젝트 README입니다. 실행 명령이나 버전이 바뀌면 이 문서도 같은 PR에서 갱신하는 것을 권장합니다.